Manual de programación

Edelstahl ERP

Documentación técnica para el programador
Prácticas Profesionalizantes III
Autor: Dante Alberto Luis Orihuela Schlögl - Legajo B00000555-VCO
Versión 1.2

Tabla de contenido

- 1. Introducción
  - 1.1. Tecnologías
- 2. Arquitectura
  - 2.1. Estructura de la solución
  - 2.2. Bases de datos
  - 2.3. Modos de ejecución
- 3. Patrones de diseño
- 4. Hilos y concurrencia
  - 4.1. Monitor de stock: hilo explícito (CU28)
  - 4.2. Importación de datos: tarea con progreso (CU33)
- 5. Seguridad
- 6. Idiomas
- 7. Compilar, ejecutar y probar
  - 7.1. Compilar
  - 7.2. Pruebas automáticas
  - 7.3. Generar el instalador
  - 7.4. Configuración
- 8. Cómo agregar una función
- 9. Referencia de clases
  - 9.1. Espacio de nombres Edelstahl.BLL.DTOs
  - 9.2. Espacio de nombres Edelstahl.BLL.Exceptions.Base
  - 9.3. Espacio de nombres Edelstahl.BLL.Exceptions.Commercial
  - 9.4. Espacio de nombres Edelstahl.BLL.Infrastructure
  - 9.5. Espacio de nombres Edelstahl.BLL.Produccion
  - 9.6. Espacio de nombres Edelstahl.BLL.Services
  - 9.7. Espacio de nombres Edelstahl.BLL.Validators
  - 9.8. Espacio de nombres Edelstahl.DAL.Backup
  - 9.9. Espacio de nombres Edelstahl.DAL.EntityFramework
  - 9.10. Espacio de nombres Edelstahl.DAL.Factory
  - 9.11. Espacio de nombres Edelstahl.DAL.Implementations.EntityFramework
  - 9.12. Espacio de nombres Edelstahl.DAL.Implementations.Memory
  - 9.13. Espacio de nombres Edelstahl.DAL.Implementations.SqlServer
  - 9.14. Espacio de nombres Edelstahl.DAL.Interfaces

- 9.15. Espacio de nombres Edelstahl.DAL.Store
- 9.16. Espacio de nombres Edelstahl.DAL.Tools
- 9.17. Espacio de nombres Edelstahl.Domain.Catalogo
- 9.18. Espacio de nombres Edelstahl.Domain.Comercial
- 9.19. Espacio de nombres Edelstahl.Domain.Common
- 9.20. Espacio de nombres Edelstahl.Domain.Finanzas
- 9.21. Espacio de nombres Edelstahl.Domain.Integrity
- 9.22. Espacio de nombres Edelstahl.Domain.Security
- 9.23. Espacio de nombres Edelstahl.Services.Configuration
- 9.24. Espacio de nombres Edelstahl.Services.Cryptography
- 9.25. Espacio de nombres Edelstahl.Services.Documents
- 9.26. Espacio de nombres Edelstahl.Services.Export
- 9.27. Espacio de nombres Edelstahl.Services.Integrity
- 9.28. Espacio de nombres Edelstahl.Services.Localization
- 9.29. Espacio de nombres Edelstahl.Services.Security
- 9.30. Espacio de nombres Edelstahl.WinApp
- 9.31. Espacio de nombres Edelstahl.WinApp.Infrastructure
- 9.32. Espacio de nombres Edelstahl.WinApp.Modulos
- 9.33. Espacio de nombres Edelstahl.WinApp.Properties
- 9.34. Espacio de nombres Edelstahl.WinApp.Ui

1. Introducción

Este manual está dirigido a quien tenga que mantener o ampliar **Edelstahl ERP**. Explica cómo está organizado el código, qué patrones usa, cómo se manejan los hilos, la seguridad y los idiomas, cómo compilar y probar, y cómo agregar una función. Al final tiene la referencia de clases, generada a partir de los comentarios de documentación XML del propio código.

Para el uso del programa ver el *Manual de usuario*; para instalarlo, el *Manual de instalación*.

1.1. Tecnologías

Componente	Tecnología
Lenguaje y plataforma	C# 7.3 sobre .NET Framework 4.8
Interfaz	Windows Forms, con componentes propios (PanelLista, Dialogo, Estilo)
Acceso a datos del negocio	Entity Framework 6 con repositorios y unidad de trabajo
Acceso a datos de servicios	ADO.NET (SqlClient) para usuarios, roles, bitácora y respaldos
Base de datos	SQL Server Express (probado con SQL Server 2022) o LocalDB
PDF	QuestPDF

Mapa de la hoja de ruta	WebView2 con una página local

2. Arquitectura

El sistema está dividido en capas. Cada capa solo conoce a las que están debajo; la interfaz nunca toca la base de datos.

2.1. Estructura de la solución

Proyecto	Capa	Qué contiene
Edelstahl.Domain	Dominio	Entidades y reglas propias de cada una (Cliente, Pedido, Producto, Factura, Usuario...), enumeraciones, la matriz de permisos y las excepciones de negocio.
Edelstahl.DAL	Acceso a datos	Contexto de Entity Framework, repositorios (interfaces públicas e implementaciones internas en memoria, EF y SQL Server), unidad de trabajo, fábrica con reflexión, SqlHelper (ADO.NET), instalador del esquema, respaldos y verificación de integridad.
Edelstahl.BLL	Lógica de negocio	Servicios con las reglas de cada módulo, DTOs, autorización, mensajes, numeración, el patrón State de las órdenes, el monitor de stock, la verificación de integridad, los respaldos y la recuperación.
Edelstahl.Services	Transversal	Cifrado AES, hash de contraseñas, idiomas (recursos .resx), documentos PDF, exportadores CSV y XML, dígitos verificadores y modo de ejecución.
Edelstahl.WinApp	Interfaz	Formularios y módulos de Windows Forms.
Edelstahl.Test	Pruebas	Ejecutable de pruebas automáticas que recorre los casos de uso.

Las dependencias van así: `WinApp → BLL → DAL → Domain`, y `Services` lo usan todas las capas.

2.2. Bases de datos

Hay dos bases:

- **EdelstahlNegocio**: 41 tablas del negocio (clientes, productos, stock, pedidos, producción, compras, finanzas y logística). El esquema lo administra Entity Framework.
- **EdelstahlServicios**: roles, usuarios, bitácora y registro de respaldos, accedidas con ADO.NET a través de SqlHelper.

Si las bases no existen, el instalador de esquema (`InicializadorNegocio` y `EsquemaServicios`) las crea al iniciar, y no pisa lo que ya hay. Los scripts SQL equivalentes están en la carpeta

`Scripts`: `01 y 02 crean las tablas; 03 y 04 cargan los datos; 05 recupera el acceso de un usuario que olvidó su contraseña (deja una contraseña provisoria y lo obliga a elegir una propia). La tabla Usuarios también tiene PreguntaSeguridad, RespuestaHash y RespuestaSalt (la respuesta se guarda como hash); el programa agrega esas columnas al arrancar si faltan.`

El modelo de datos completo, con los diagramas entidad-relación, está en el modelo de Enterprise Architect (carpeta *6-DER*).

2.3. Modos de ejecución

`ApplicationMode` y `FactoryDataAccess` deciden de dónde salen los datos:

- **Demostración:** repositorios en memoria. No hay base de datos.
- **SQL Server:** repositorios de Entity Framework y ADO.NET.

El resto del código no sabe en qué modo está: pide un `INegocioStore` a la fábrica y trabaja con él. Así las mismas pruebas sirven para los dos modos.

3. Patrones de diseño

Patrón	Dónde	Para qué
Repository	IGenericRepository y los repositorios por entidad	Aislar el acceso a datos de la lógica de negocio.
Unit of Work	INegocioStore / IUnidadDeTrabajo	Agrupar varios cambios en una sola transacción.
Factory	FactoryDataAccess	Elegir por reflexión la implementación (memoria o SQL Server) según el modo, validando que exista, sea concreta, implemente la interfaz y tenga constructor sin parámetros.
State	EstadosOrden, IEstadoOrden y los estados de la orden de fabricación	Cada estado (pendiente, en proceso, en control de calidad, finalizada, cancelada) decide qué transiciones permite.
Template Method	DocumentoPdfBase y los PDF; ExportadorBase y los exportadores	Definir el esqueleto del documento o de la exportación y dejar que cada clase complete los pasos.
Observer	Evento AlertasActualizadas del monitor de stock; cambio de idioma	Avisar a la interfaz cuando algo cambia, sin que el origen conozca a la interfaz.
Singleton	SesionActual	Un único usuario conectado accesible desde cualquier capa.

Facade	Servicios transversales (LanguageService, CryptographyService, PasswordHasher)	Dar un punto de entrada simple a tareas complejas.

3.1. Factory con reflexión

FactoryDataAccess no decide con una cadena de if: guarda, para cada modo de ejecución y cada interfaz, el nombre de la clase que la implementa y la crea por reflexión (Assembly.GetType y GetConstructor). Antes de crearla valida cuatro cosas, y cada fallo lanza una ConfiguracionInvalidaException con un mensaje claro:

- que la clase exista en el ensamblado;
- que sea concreta (no abstracta ni una interfaz);
- que implemente la interfaz pedida;
- que tenga un constructor sin parámetros (puede ser interno).

Los nombres por defecto están en una tabla dentro de la fábrica y se pueden cambiar, sin recompilar, con una clave de App.config con la forma Repositorio.[Modo].[Interfaz], por ejemplo Repositorio.SqlServer.IClienteRepository. CrearDesdeNombre<T>(nombre) es público para poder probar las validaciones.

3.2. SqlHelper

Todo el ADO.NET de la aplicación pasa por la clase interna SqlHelper (Edelstahl.DAL.Tools): ningún repositorio abre una SqlConnection ni arma un SqlCommand por su cuenta.

- EjecutarNoQuery, EjecutarEscalar<T>, Consultar y ConsultarUno ejecutan una sentencia con parámetros y cierran la conexión. Consultar recibe una función que convierte cada fila en un objeto.
- EnTransaccion y Transaccion agrupan varias sentencias en una transacción que se confirma o se revierte entera.
- Parametro, ONulo, Texto, GuidONulo y FechaONula arman los parámetros y convierten los nulos de la base.
- El enumerado BaseDeDatos (Negocio, Servicios, Master) indica contra cuál se ejecuta; la cadena de conexión sale de SqlServerConnection.

Toda SqlException se envuelve en una AccesoADatosException, de modo que las capas superiores no dependen de System.Data.SqlClient.

3.3. Excepciones de negocio

Las excepciones de negocio están en Edelstahl.Domain.Exceptions y derivan de BusinessRuleException. La interfaz (Estilo.Ejecutar) captura esa clase base y muestra el mensaje tal cual al usuario; cualquier otra excepción se muestra como Error inesperado. ArgumentNullException queda solo para parámetros nulos, que son errores de programación.

- **ReglaDeNegocioException:** una regla del negocio no se cumple (por ejemplo, confirmar un pedido sin stock).

- **AccesoNoAutorizadoException:** el rol no tiene el permiso.
- **DatoInvalidoException:** un dato falta o no es válido.
- **RegistroNoEncontradoException y RegistroDuplicadoException:** el registro buscado no existe o ya existe.
- **ClienteDuplicadoException y CUITInvalidoException:** reglas del cliente; llevan el CUIT.
- **AccesoADatosException:** falló la base de datos (la crea SqlHelper).
- **RespaldoException:** falló un respaldo o una restauración.
- **ConfiguracionInvalidaException:** la fábrica no pudo crear una clase o falta una configuración.
- **IntegridadVulneradaException:** un dígito verificador no coincide.

Los mensajes pasan por LanguageService (Translate y Format), con la frase en inglés como clave, así que salen en el idioma del usuario.

3.4. Encapsulación

Cada capa expone solo lo que otras capas necesitan; el resto es internal.

- **DAL:** son públicas FactoryDataAccess y las interfaces (IClienteRepository, IRespaldoRepository, INegocioStore...). Las implementaciones en memoria, EF y SQL Server, el contexto de Entity Framework, el esquema, SqlHelper, el respaldo y el verificador de integridad son internal.
- **BLL:** los servicios y los DTO son públicos, porque los usa la interfaz. Son internal los validadores, el motor de inventario, los estados de la orden, LectorCsv, Numeracion y Direcciones.
- **Services:** son internal CryptographySettings, AesEncryptionService y ClienteDvhService.
- Edelstahl.Test ve los tipos internos con InternalsVisibleTo (en los AssemblyInfo de DAL, BLL y Services), para probarlos sin hacerlos públicos.

Así la interfaz no puede saltearse la lógica: desde WinApp no se puede crear un repositorio ni ejecutar SQL; solo se pasa por los servicios de la BLL.

4. Hilos y concurrencia

El sistema usa dos mecanismos distintos de ejecución en segundo plano. Se usaron ambos a propósito, para mostrar el hilo explícito y la tarea del grupo de hilos.

4.1. Monitor de stock: hilo explícito (CU28)

`MonitorDeStock` crea un `Thread` propio, marcado como de segundo plano y con nombre `"MonitorDeStock"`. En un ciclo, cada intervalo (mínimo 5 segundos), llama a `InventarioService.ObtenerAlertas()`, compara con la cantidad anterior y, solo si cambió, dispara el evento `AlertasActualizadas`.

- **Detener:** un `ManualResetEventSlim` despierta al hilo y `Detener()` espera hasta 2 segundos con `Join`.
- **Exclusión mutua:** un `lock` protege la creación y la detención del hilo.

- **Contador sin bloqueo:** `Interlocked.Exchange` actualiza la última cantidad de alertas.
- **Interfaz:** el evento llega en el hilo del monitor; la ventana principal usa `BeginInvoke` para volver al hilo de la interfaz antes de actualizar el contador.

4.2. Importación de datos: tarea con progreso (CU33)

La importación de clientes y productos corre con `Task.Run`, informa el avance con `IProgress<int>` y, al terminar, vuelve a la interfaz con `BeginInvoke`. La ventana no se bloquea y la barra de avance se actualiza sola.

> **Importante:** Ninguna clase de dominio ni de acceso a datos crea hilos. Solo lo hacen el monitor de stock y la importación; así el resto del código no necesita pensar en concurrencia.

5. Seguridad

- **Contraseñas:** PBKDF2 con sal e iteraciones guardadas junto al hash (`PasswordHasher`). Nunca se guarda la contraseña.
- **Bloqueo:** tras varios intentos fallidos el usuario se bloquea por un tiempo (`Usuario.RegistrarAccesoFallido`).
- **Cifrado:** el CUIT de los clientes se cifra con AES (`AesEncryptionService`); la clave está en `App.config`. Es una configuración académica: en producción iría en un gestor de secretos.
- **Integridad:** dígitos verificadores horizontal (por fila) y vertical (por tabla), `ClienteDvhService` y `ClienteDvvService`, para detectar alteraciones directas en la base. Qué pasa cuando se detecta una alteración: ver 5.1.
- **Permisos:** cada servicio llama a `Autorizacion.Exigir(permiso)` antes de operar; la matriz de permisos por rol está en `MatrizPermisos`. El administrador puede agregar o quitar permisos a un usuario puntual (ver 5.2).
- **Bitácora:** toda acción relevante y todo error se registran con `BitacoraService`.

5.1. Integridad, respaldos y recuperación

Al iniciar, Program.VerificarIntegridad() llama a IntegridadService.VerificarYRegistrar. El verificador (VerificadorIntegridad, en la DAL) recalcula el DVH de cada cliente (SHA-256 de sus campos) y el DVV de la tabla (hash del Id y del DVH de todos los registros) y los compara con los guardados. Cambiar solo datos rompe el DVH; borrar o agregar filas rompe el DVV.

Si hay una inconsistencia, la bitácora recibe una entrada (módulo Integridad, acción Alteracion detectada, resultado Fallido, tipo Seguridad) que dice qué registros cambiaron y cuándo, y en lugar del inicio de sesión se abre FrmRecuperacionIntegridad. Esa pantalla pide usuario y contraseña (RecuperacionService.Autenticar exige el permiso Permiso.RespaldoAdministrar), lista los respaldos de la tabla dbo.Respaldos y restaura el elegido.

- **Respaldar:** RespaldoService usa RespaldoDatabaseService, que ejecuta BACKUP DATABASE con COPY_ONLY y CHECKSUM, y verifica el archivo con RESTORE VERIFYONLY. Si SQL Server no puede escribir en la carpeta elegida (error 3201), escribe en su propia carpeta y la aplicación copia el archivo al destino. El nombre del archivo lleva milisegundos para que dos respaldos seguidos no se pisen.

- **Sano:** un respaldo se marca como sano solo si la verificación de integridad era correcta justo antes de crearlo. El respaldo automático (uno por día, se conservan los 10 últimos) solo se hace con datos sanos.
- **Restaurar:** la base pasa a SINGLE_USER con ROLLBACK IMMEDIATE, se ejecuta RESTORE con REPLACE y CHECKSUM (con MOVE según FILELISTONLY) y se limpian los pools de conexiones. Antes se guarda una copia del estado actual (origen AntesDeRestaurar). Solo se restaura la base de negocio: la de servicios guarda usuarios, bitácora e historial de respaldos.
- **Recalcular:** RecuperacionService.RecalcularDigitos acepta los datos actuales como correctos y deja una entrada de auditoría en la bitácora.

La carpeta de respaldos y la casilla del respaldo automático se guardan en respaldos.config, dentro de %LOCALAPPDATA%\Edelstahl ERP (o de la carpeta indicada por la variable de entorno EDELSTAHL_DATOS_LOCALES, que usan las pruebas para no tocar la configuración real).

5.2. Permisos individuales

AdemÃ¡s de los permisos de su rol, el administrador puede agregar o quitar permisos a una persona puntual. Esas excepciones se guardan en la tabla PermisosUsuario de la base de servicios, una fila por usuario y permiso (Ã-ndice Ãºnico), con la clase PermisoIndividual: UsuarioId, Permiso, Concedido, FechaCambio y Administrador (quiÃ©n hizo el cambio).

- PermisosEfectivos.Tiene(usuario, permiso) calcula el permiso real: los del rol (MatrizPermisos), mÃ¡s los agregados, menos los quitados. SesionActual.Tiene lo usa, de modo que Autorizacion.Exigir ya tiene en cuenta las excepciones y ningÃºn servicio cambia.
- Las excepciones de cada usuario se leen una sola vez y quedan en memoria (PermisosEfectivos.Invalidar las descarta). Por eso un cambio se aplica la prÃ³xima vez que esa persona inicia sesiÃ³n.
- AdministracionService.ObtenerPermisosDeUsuario devuelve, por cada permiso, si lo da el rol, si estÃ¡ efectivo y si hay una excepciÃ³n individual (PermisoDeUsuarioDto).
- AdministracionService.CambiarPermisosDeUsuario recibe cÃ³mo debe quedar el usuario y guarda solo lo que difiere de su rol; lo que vuelve a coincidir con el rol borra la excepciÃ³n. Devuelve la cantidad de cambios. RestablecerPermisos borra todas las excepciones del usuario.
- Solo un administrador puede hacerlo (ExigirAdministrador). Administrar usuarios, roles y respaldos (MatrizPermisos.SoloAdministrador) no se puede dar a quien no sea administrador, y a un administrador no se le quita ningÃºn permiso. Si se cambia el rol de un usuario, sus excepciones se borran.
- Cada cambio queda en la bitÃ¡cora con las acciones Permiso agregado, Permiso quitado, Permiso restablecido y Permisos restablecidos.
- La pantalla es FrmPermisosUsuario (botÃ³n Permisos de la pestaÃ±a Usuarios de AdministraciÃ³n). El acceso a datos estÃ¡ detrÃ¡s de IPermisoUsuarioRepository, con las implementaciones PermisoUsuarioRepositorySqlServer y PermisoUsuarioRepositoryMemory que crea la Factory segÃºn el modo.

6. Idiomas

Los textos de la interfaz se escriben en inglés en el código y se traducen al mostrarlos con `Tr.T("texto")` (interfaz) o `Mensajes.T` y `Mensajes.Regla` (reglas de negocio). La clave es la propia frase en inglés.

Las traducciones están en archivos de recursos (.resx) del proyecto Edelstahl.Services, carpeta Localization: Textos.resx (neutro, en inglés) y un Textos.[cultura].resx por idioma (es-AR, es-ES, en-GB, de-DE, pt-BR y ru-RU). Cada entrada tiene como nombre la frase en inglés y como valor la traducción. Para agregar un texto hay que agregarlo en el .resx de cada idioma; si falta en uno, se muestra en inglés. Visual Studio compila cada .resx como un ensamblado satélite (por ejemplo es-AR\Edelstahl.Services.resources.dll) que se instala junto al ejecutable.

LanguageService lee los recursos con ResourceManager.GetResourceSet y arma el diccionario de cada idioma combinando el neutro, el idioma padre (es para es-AR) y el elegido, de modo que lo que falta cae al idioma más general. Guarda un diccionario por idioma en memoria. Tr.T, Mensajes.T y Mensajes.Regla siguen funcionando igual.

El programa arranca siempre en inglés y aplica el idioma del usuario al iniciar sesión. `LanguageService` notifica el cambio a las pantallas abiertas (patrón Observer).

7. Compilar, ejecutar y probar

7.1. Compilar

1. Abrir `Edelstahl.slnx` con Visual Studio 2022 o posterior. Los paquetes NuGet se restauran solos.
2. Dejar **Edelstahl.WinApp** como proyecto de inicio.
3. Compilar (Ctrl+Mayús+B) y ejecutar con F5.

También se puede compilar desde la línea de comandos con `MSBuild Edelstahl.slnx -p:Configuration=Release`.

7.2. Pruebas automáticas

```
Edelstahl.Test.exe [cifrado] [demo] [sql] [pantallas] [pantallassql] [prep]
[catalogo <archivo.csv>] [recuperacion <carpeta>]
```

- `demo` y `sql` recorren los casos de uso de punta a punta, en memoria o contra SQL Server.
- `pantallas` abre cada módulo y cada diálogo principal.
- `catalogo` prueba la carga masiva de productos, las reglas del catálogo, el aumento de precios y las órdenes de compra.

Para no tocar las bases reales, definir antes las variables de entorno `EDELSTAHL_NEGOCIO_DB` y `EDELSTAHL_SERVICIOS_DB` con nombres de prueba. Las pruebas SQL se niegan a correr si la base de negocio se llama EdelstahlNegocio. Las preferencias del usuario (idioma y respaldos) se redirigen a una carpeta temporal con la variable EDELSTAHL_DATOS_LOCALES, así que las pruebas no tocan la configuración real.

7.3. Generar el instalador

El instalador es un proyecto de *Visual Studio Installer Projects* (`Edelstahl.Installer\Edelstahl.Installer.vdproj`). Genera `Edelstahl ERP.msi` y `Setup.exe` con la aplicación compilada en Release, los idiomas, los datos de ejemplo y los scripts SQL.

1. Instalar la extensión gratuita *Microsoft Visual Studio Installer Projects* (Extensiones / Administrar extensiones).
2. Compilar la solución en Release: `MSBuild Edelstahl.slnx -p:Configuration=Release`.
3. Compilar el instalador: `devenv.com Edelstahl.Installer\Instalador.sln /Build Release`. Ese archivo contiene solo el instalador y lo marca como compilable; dentro de Visual Studio alcanza con clic derecho sobre *Edelstahl.Installer* y *Compilar*.
4. Los archivos quedan en `Edelstahl.Installer\Release`. Antes hay que compilar Edelstahl.WinApp en Release: los idiomas salen como ensamblados satélite (carpetas es-AR, es-ES, en-GB, de-DE, pt-BR y ru-RU dentro de bin\Release) y el proyecto del instalador los lista uno por uno. Después de generar el MSI hay que correr Edelstahl.Installer\parchear-icono-msi.ps1 con -msi <ruta del MSI> y -parchear, para que el icono de los accesos directos se vea.

El icono del programa, un yunque, está en `Edelstahl.WinApp\Resources\Icono\edelstahl.ico`. Se usa como icono del ejecutable, de las ventanas, de los accesos directos y del programa en la lista de aplicaciones instaladas.

El instalador pide .NET Framework 4.8 como condición de instalación, copia los archivos en `C:\Program Files (x86)\Edelstahl\Edelstahl ERP\` y crea los accesos directos del Escritorio y del menú Inicio. No crea las bases de datos: las crea el programa al iniciar, o se cargan con los scripts.

> **Nota:** Si se agrega un archivo nuevo a la aplicación hay que agregarlo también al proyecto del instalador; el proyecto lista cada archivo de `bin\Release` de forma explícita.

7.4. Configuración

Clave de App.config	Para qué sirve
SqlServerInstancia	Instancia de SQL Server. Vacía: busca localhost\SQLEXPRESS y luego LocalDB.
BaseNegocio	Nombre de la base del negocio (EdelstahlNegocio).
BaseServicios	Nombre de la base de servicios (EdelstahlServicios).
AesEncryptionSecret	Clave del cifrado AES del CUIT.

Las mismas opciones se pueden fijar con las variables de entorno `EDELSTAHL_SQL_SERVER`, `EDELSTAHL_NEGOCIO_DB` y `EDELSTAHL_SERVICIOS_DB`, que tienen prioridad.

8. Cómo agregar una función

Ejemplo: agregar una operación nueva en un módulo existente.

1. **Dominio:** si hace falta un dato nuevo, agregá la propiedad a la entidad en `Edelstahl.Domain` y la regla propia de la entidad.
2. **Datos:** agregá la configuración de la propiedad en el contexto de Entity Framework y, si es una tabla nueva, el repositorio. Actualizá el script de esquema.
3. **Lógica:** agregá el método al servicio, llamando primero a `Autorizacion.Exigir` y escribiendo las reglas con `Mensajes.Regla`. Registrá la acción en la bitácora.
4. **Interfaz:** agregá el botón en el módulo con `lista.Agregar("Texto en inglés", acción, ..., Permiso.X)` y los diálogos con `Dialogo.Pedir`.
5. **Idiomas:** agregá la frase nueva en el Textos.[cultura].resx de cada idioma (Edelstahl.Services\Localization).
6. **Prueba:** sumá la verificación a `Edelstahl.Test` y ejecutala en modo demo y en modo sql.
7. **Documentación:** comentá el código con `///` y actualizá el modelo.

> **Nota:** Los comentarios `///` alimentan la referencia de este manual. Para regenerarla, compilar la solución (se crean los archivos `.xml`) y volver a ejecutar el generador de manuales.

9. Referencia de clases

Generada a partir de los comentarios de documentación del código. Los tipos y miembros sin comentario no aparecen; el modelo de Enterprise Architect (carpeta *4-Diagrama de Clases*) los muestra completos. Desde la versión 1.2 las implementaciones de la DAL, los validadores y otros ayudantes son internal (ver 3.4): la referencia los sigue describiendo porque documentan el código, pero solo se pueden usar desde su propio proyecto y desde Edelstahl.Test. Las clases de recuperación, integridad y respaldo se explican en 5.1; la fábrica, SqlHelper y las excepciones, en 3.1 a 3.3.

9.1. Espacio de nombres Edelstahl.BLL.DTOs

Capa: **Lógica de negocio**. 8 tipo(s) con documentación.

Tipo	Descripción
`CobroPendienteDto`	Factura con saldo por cobrar, con lo abonado y la forma de pago prevista.
`DetallePresupuestoDto`	Contiene la información de un renglón del presupuesto que se mostrará en la grilla de productos.
`EnvioHistorialDto`	Una fila de la bitacora de envios: un remito con su recorrido y resultado.

`EventoHistorialDto`	Un paso del historial de una orden de fabricacion o de un envio, con quien lo hizo.
`FaltanteDto`	Producto que se pidio en presupuestos y no alcanza en existencias.
`OrdenPorAceptarDto`	Orden de fabricacion que espera que Produccion la acepte, con los datos de la venta y del pago.
`OrdenPreparacionDto`	Pedido facturado que el deposito debe preparar antes de despacharlo.
`PresupuestoSeleccionDto`	Contiene la información necesaria para mostrar y seleccionar un presupuesto.

9.1.1. CobroPendienteDto

Factura con saldo por cobrar, con lo abonado y la forma de pago prevista.

9.1.2. DetallePresupuestoDto

Contiene la información de un renglón del presupuesto que se mostrará en la grilla de productos.

9.1.3. EnvioHistorialDto

Una fila de la bitacora de envios: un remito con su recorrido y resultado.

9.1.4. EventoHistorialDto

Un paso del historial de una orden de fabricacion o de un envio, con quien lo hizo.

9.1.5. FaltanteDto

Producto que se pidio en presupuestos y no alcanza en existencias.

9.1.6. OrdenPorAceptarDto

Orden de fabricacion que espera que Produccion la acepte, con los datos de la venta y del pago.

9.1.7. OrdenPreparacionDto

Pedido facturado que el deposito debe preparar antes de despacharlo.

9.1.8. PresupuestoSeleccionDto

Contiene la información necesaria para mostrar y seleccionar un presupuesto.

9.2. Espacio de nombres Edelstahl.Domain.Exceptions (clases base)

Capa: **Dominio**. 3 tipo(s) con documentación.

Tipo	Descripción
`AccesoNoAutorizadoException`	Se produce cuando el usuario no tiene permiso para la operación.
`BusinessRuleException`	Excepción base para las reglas de negocio de Edelstahl.
`ReglaDeNegocioException`	Se produce cuando una operación incumple una regla del negocio. El mensaje ya está traducido al idioma activo.

9.2.1. AccesoNoAutorizadoException

Se produce cuando el usuario no tiene permiso para la operación.

Hereda de o implementa: `BusinessRuleException`.

9.2.2. BusinessRuleException

Excepción base para las reglas de negocio de Edelstahl.

Hereda de o implementa: `Exception`.

9.2.3. ReglaDeNegocioException

Se produce cuando una operación incumple una regla del negocio. El mensaje ya está traducido al idioma activo.

Hereda de o implementa: `BusinessRuleException`.

9.3. Espacio de nombres Edelstahl.Domain.Exceptions (clases comerciales)

Capa: **Dominio**. 2 tipo(s) con documentación.

Tipo	Descripción
`ClienteDuplicadoException`	Se produce cuando ya existe un cliente registrado con el CUIT informado.
`CUITInvalidoException`	Se produce cuando el CUIT ingresado no cumple con las reglas establecidas.

9.3.1. ClienteDuplicadoException

Se produce cuando ya existe un cliente registrado con el CUIT informado.

Hereda de o implementa: `BusinessRuleException`.

9.3.2. CUITInvalidoException

Se produce cuando el CUIT ingresado no cumple con las reglas establecidas.

Hereda de o implementa: `BusinessRuleException`.

9.4. Espacio de nombres Edelstahl.BLL.Infrastructure

Capa: **Lógica de negocio**. 7 tipo(s) con documentación.

Tipo	Descripción
`Autorizacion`	Control de acceso basado en roles. Las operaciones sensibles llaman a Exigir antes de modificar datos; los intentos denegados quedan en la bitácora.
`Direcciones`	Reconoce las direcciones que en realidad no existen (vacias o con el texto de relleno de la carga masiva).
`Etiquetas`	Texto legible (y traducible) de los estados y tipos del sistema. La clave de traducción es el texto en inglés.
`LectorCsv`	Lector de archivos CSV con separador configurable. Admite campos entre comillas, comillas duplicadas y saltos de línea dentro de un campo. Detecta si el archivo está en UTF-8 o en Windows-1252.
`Mensajes`	Mensajes de negocio traducibles. La clave es el texto en inglés; si el idioma activo no tiene traducción se muestra ese mismo texto.
`Numeracion`	Genera números correlativos con el formato PREFIJO-AAAA-NNNNNN.
`Referencias`	Datos de referencia (clientes y pedidos) que los módulos de producción, finanzas y logística muestran junto a sus propios registros.

9.4.1. Autorizacion

Control de acceso basado en roles. Las operaciones sensibles llaman a Exigir antes de modificar datos; los intentos denegados quedan en la bitácora.

9.4.2. Direcciones

Reconoce las direcciones que en realidad no existen (vacias o con el texto de relleno de la carga masiva).

9.4.3. Etiquetas

Texto legible (y traducible) de los estados y tipos del sistema. La clave de traducción es el texto en inglés.

Miembro	Tipo	Descripción
`TonosPorTexto`	Método	Texto ya traducido de cada estado -> 1 (bien), 2 (en curso o atencion) o 3 (problema). Sirve para pintar las celdas de estado en las tablas.

9.4.4. LectorCsv

Lector de archivos CSV con separador configurable. Admite campos entre comillas, comillas duplicadas y saltos de línea dentro de un campo. Detecta si el archivo está en UTF-8 o en Windows-1252.

Miembro	Tipo	Descripción
`Columna(System.String[],System.String[])`	Método	Devuelve el índice de la primera columna cuyo encabezado coincide, sin distinguir mayúsculas ni acentos.

9.4.5. Mensajes

Mensajes de negocio traducibles. La clave es el texto en inglés; si el idioma activo no tiene traducción se muestra ese mismo texto.

Miembro	Tipo	Descripción
`Regla(System.String,System.Object[])`	Método	Crea una excepción de regla de negocio con el mensaje traducido.

9.4.6. Numeracion

Genera números correlativos con el formato PREFIJO-AAAA-NNNNNN.

Miembro	Tipo	Descripción
`SiguienteFiscal(System.Int32,Generic.IEnumerable{System.String})`	Método	Numeración fiscal: punto de venta de cuatro cifras y número de ocho.

9.4.7. Referencias

Datos de referencia (clientes y pedidos) que los módulos de producción, finanzas y logística muestran junto a sus propios registros.

9.5. Espacio de nombres Edelstahl.BLL.Produccion

Capa: **Lógica de negocio**. 2 tipo(s) con documentación.

Tipo	Descripción
`EstadoEnProceso`	Estado de la orden mientras el taller trabaja. Se pueden registrar avances, mandarla a control de calidad o cancelarla.

`IEstadoOrden`	Patrón State aplicado a la orden de fabricación: cada estado decide qué operaciones admite y a qué estado pasa la orden. Así las etapas respetan el orden definido y no pueden omitirse.

9.5.1. EstadoEnProceso

Hereda de o implementa: `EstadoOrdenBase`.

Miembro	Tipo	Descripción
`AvanceDe(Produccion.OrdenFabricacion)`	Método	Avance de fabricación de la orden: el mayor porcentaje registrado (0 si todavía no se cargó ninguno).

9.5.2. IEstadoOrden

Patrón State aplicado a la orden de fabricación: cada estado decide qué operaciones admite y a qué estado pasa la orden. Así las etapas respetan el orden definido y no pueden omitirse.

9.6. Espacio de nombres Edelstahl.BLL.Services

Capa: **Lógica de negocio**. 27 tipo(s) con documentación.

Tipo	Descripción
`AdministracionService`	Administración de usuarios y roles. Solo la usa quien tiene permiso de administración; cada operación queda en la bitácora.
`BitacoraService`	Registra en la bitácora lo que pasa en el sistema (información, auditoría, seguridad y errores) y permite consultar lo ocurrido con una entidad.
`CargaInicialService`	Datos mínimos para operar (roles, depósito y medios de pago) y datos de ejemplo (catálogo, clientes, proveedores, vehículos y choferes).
`CatalogoService`	Catálogo maestro: categorías, productos y materiales, con sus precios y existencias.
`ClienteService`	Coordina las operaciones y reglas de negocio relacionadas con los clientes.
`ComprasService`	Proveedores, órdenes de compra y recepción de materiales.
`ConfirmarPedidoService`	Coordina el proceso completo de confirmación de un pedido. Mantiene las selecciones realizadas durante el asistente.

`DemoAuthenticationService`	Autentica al usuario temporal utilizado cuando Edelstahl funciona en modo demostración. No utiliza SQL Server y no conserva la contraseña en texto plano.
`DevolucionService`	Logística inversa: devoluciones de mercadería, inspección, reingreso al inventario y base para la nota de crédito.
`DocumentosService`	Arma los datos de cada documento imprimible y genera su PDF.
`FacturacionService`	Emisión de facturas A y B y de notas de crédito. Cada comprobante genera su movimiento en la cuenta corriente y actualiza la deuda del cliente.
`FaltantesService`	Calcula que productos faltan para poder vender: compara lo que piden los presupuestos con las existencias disponibles (stock fisico menos lo reservado para otros pedidos).
`FinanzasService`	Cobros a clientes, pagos a proveedores y consulta de las cuentas corrientes.
`ImportacionService`	Importación de clientes y productos desde archivos CSV (formato de exportación de tienda en línea, separados por punto y coma).
`InventarioMotor`	Reglas de existencias, reservas y movimientos. Opera siempre dentro de la unidad de trabajo recibida, de modo que cada servicio decide qué operaciones se confirman o se revierten juntas.
`InventarioService`	Consulta de existencias, movimientos, reservas y alertas de reposición.
`LogisticaService`	Vehículos, choferes, remitos, hojas de ruta y registro de entregas.
`MonitorDeStock`	Revisa periódicamente el stock en un hilo propio y avisa cuando cambia la lista de productos por debajo del mínimo (patrón Observer).
`PedidoService`	Coordina la creación, validación y persistencia de los pedidos comerciales.
`PreparacionService`	Deja un pedido en condiciones de despacharse: reserva el stock que falta, ingresa la mercadería faltante si se pide y completa los materiales de una orden de fabricación.
`PresupuestoService`	Coordina las operaciones relacionadas con presupuestos.
`ProduccionService`	Órdenes de fabricación: generación, reserva de materiales, avance, consumos, control de calidad y finalización.

`ReporteService`	Indicadores y reportes de gestión. Todos los importes se expresan en pesos; los pedidos en dólares se convierten con su tipo de cambio.
`RespaldoService`	Coordina el respaldo de las bases de negocio y servicios.
`SeguimientoService`	Consulta del estado de un pedido en todas sus etapas: comercial, productiva, de facturación y logística.
`SesionActual`	Mantiene los datos del usuario autenticado durante la ejecución.
`UsuarioService`	Coordina el registro, autenticacion y administracion de los usuarios de Edelstahl ERP.

9.6.1. AdministracionService

Administración de usuarios y roles. Solo la usa quien tiene permiso de administración; cada operación queda en la bitácora.

Miembro	Tipo	Descripción
`ModificarDatos(System.Guid,System.String,System.String)`	Método	El administrador corrige el nombre completo y el correo de un usuario.

9.6.2. BitacoraService

Miembro	Tipo	Descripción
`ObtenerDeEntidad(System.Guid)`	Método	Registros de la bitacora de una entidad (orden, remito...), mas antiguos primero.
`Consultar(System.Nullable{System.DateTime},System.Nullable{System.DateTime},System.String,System.String,System.String,System.String)`	Método	Consulta de la bitácora con filtros opcionales. Requiere permiso de lectura y la propia consulta queda registrada.

9.6.3. CargaInicialService

Datos mínimos para operar (roles, depósito y medios de pago) y datos de ejemplo (catálogo, clientes, proveedores, vehículos y choferes).

Miembro	Tipo	Descripción
`AsegurarDatosBase`	Método	Crea lo imprescindible si falta. Puede ejecutarse varias veces.

`CargarDatosDeEjemplo(System.IProgress{System.Int32})`	Método	Carga catálogo, materias primas, clientes, proveedores, vehículos y choferes de ejemplo. Requiere un usuario con permisos de administración.

9.6.4. CatalogoService

Catálogo maestro: categorías, productos y materiales, con sus precios y existencias.

Miembro	Tipo	Descripción
`ObtenerRutasDeCategoria`	Método	Devuelve el nombre completo de cada categoría (Padre > Hija).
`ObtenerFabricables`	Método	Productos activos y vendibles que se pueden fabricar a medida (base de una linea de fabricacion especial).
`AumentoMasivoPrecios(System.Nullable{System.Guid},System.Decimal,System.String)`	Método	Aplica un ajuste porcentual a los precios de una categoría completa (con sus subcategorías) o de todo el catálogo.

9.6.5. ClienteService

Coordina las operaciones y reglas de negocio relacionadas con los clientes.

9.6.6. ComprasService

Proveedores, órdenes de compra y recepción de materiales.

9.6.7. ConfirmarPedidoService

Coordina el proceso completo de confirmación de un pedido. Mantiene las selecciones realizadas durante el asistente.

9.6.8. DemoAuthenticationService

Autentica al usuario temporal utilizado cuando Edelstahl funciona en modo demostración. No utiliza SQL Server y no conserva la contraseña en texto plano.

9.6.9. DevolucionService

Logística inversa: devoluciones de mercadería, inspección, reingreso al inventario y base para la nota de crédito.

Miembro	Tipo	Descripción

`ResolverDetalle(DTOs.ResolucionDevolucion)`	Método	Inspecciona un ítem devuelto y ejecuta su resolución: reingreso al stock, reproceso, reemplazo o descarte.
`CalcularNotaCreditoSugerida(System.Guid)`	Método	Calcula el importe neto de la nota de crédito que corresponde a una devolución resuelta, valuando cada ítem al precio facturado.

9.6.10. DocumentosService

Arma los datos de cada documento imprimible y genera su PDF.

9.6.11. FacturacionService

Emisión de facturas A y B y de notas de crédito. Cada comprobante genera su movimiento en la cuenta corriente y actualiza la deuda del cliente.

Miembro	Tipo	Descripción
`EstadoVisible(Finanzas.Factura)`	Método	Una factura con saldo cuyo vencimiento pasó se muestra como vencida.

9.6.12. FaltantesService

Calcula que productos faltan para poder vender: compara lo que piden los presupuestos con las existencias disponibles (stock fisico menos lo reservado para otros pedidos).

Miembro	Tipo	Descripción
`DePresupuesto(System.Guid)`	Método	Productos que faltan para confirmar un presupuesto en particular.
`Generales`	Método	Productos que faltan para los presupuestos enviados o aceptados que siguen vigentes.

9.6.13. FinanzasService

Cobros a clientes, pagos a proveedores y consulta de las cuentas corrientes.

Miembro	Tipo	Descripción
`CodigoMedioPago(System.String)`	Método	Traduce el medio de pago escrito al registrar la seña al código del medio de pago configurado.
`ActualizarDeudaCliente(System.Guid,System.Decimal)`	Método	Fija la deuda del cliente. Pasa por el repositorio de clientes, que recalcula los dígitos verificadores de integridad.

`ObtenerCobrosPendientes`	Método	Facturas con saldo por cobrar: lo abonado (por ejemplo la sena), lo que falta y como se prevee pagar.
`ActualizarFormaDePago(System.Guid,System.String)`	Método	Anota como piensa pagar el cliente (efectivo, transferencia, cheque...) antes de que abone.

9.6.14. ImportacionService

Importación de clientes y productos desde archivos CSV (formato de exportación de tienda en línea, separados por punto y coma).

Miembro	Tipo	Descripción
`GenerarCuit(System.String)`	Método	Genera un CUIT de formato válido (dígito verificador correcto) y estable para una misma semilla. Se usa cuando el archivo no lo trae.

9.6.15. InventarioMotor

Reglas de existencias, reservas y movimientos. Opera siempre dentro de la unidad de trabajo recibida, de modo que cada servicio decide qué operaciones se confirman o se revierten juntas.

Miembro	Tipo	Descripción
`Mover(Store.IUnidadDeTrabajo,System.Guid,System.Guid,Catalogo.TipoMovimientoStock,System.Decimal,System.Nullable{System.Decimal},System.String,System.Nullable{System.Guid},System.String,System.String,System.String,System.Boolean)`	Método	Registra un movimiento que cambia la existencia física. Las reservas y liberaciones usan Reservar y LiberarReserva.
`Reservar(Store.IUnidadDeTrabajo,System.Guid,System.Decimal,System.Nullable{System.Guid},System.Nullable{System.Guid},System.String)`	Método	Reserva existencias repartiéndolas entre las ubicaciones que tengan disponible. Si no alcanza, no reserva nada.
`ConsumirReserva(Store.IUnidadDeTrabajo,Catalogo.ReservaStock,System.Decimal,Catalogo.TipoMovimientoStock,System.String,System.Nullable{System.Guid})`	Método	Consume una parte de la reserva: baja la existencia física y la reservada a la vez.

9.6.16. InventarioService

Consulta de existencias, movimientos, reservas y alertas de reposición.

Miembro	Tipo	Descripción
`ObtenerAlertas`	Método	Productos cuya existencia disponible llegó al punto de reposición.

9.6.17. LogisticaService

Vehículos, choferes, remitos, hojas de ruta y registro de entregas.

Miembro	Tipo	Descripción
`ObtenerOrdenesPreparacion`	Método	Ordenes de preparacion: pedidos ya facturados con mercaderia pendiente de enviar, con el detalle de lo que hay que preparar y si ya esta todo reservado.
`FechaComprometida(Comercial.Pedido)`	Método	Fecha de entrega pactada: fecha del pedido mas los dias del plazo ("7 dias"). Vacia si el plazo no tiene numero.
`ObtenerHistorialEnvios`	Método	Bitacora de envios: todos los remitos (abiertos y cerrados) con su recorrido y resultado.
`ObtenerEventosEnvio(System.Guid)`	Método	Todos los pasos de un envio, con el usuario que los hizo y el motivo si no se entrego o se cancelo.
`ReprogramarRemito(System.Guid,System.String)`	Método	Un envio rechazado vuelve a quedar disponible para una nueva hoja de ruta, dejando constancia del motivo.
`DespacharStock(Store.IUnidadDeTrabajo,Logistica.RemitoEntrega)`	Método	La mercadería sale del depósito al despachar: se consumen las reservas de cada línea del remito.

9.6.18. MonitorDeStock

Revisa periódicamente el stock en un hilo propio y avisa cuando cambia la lista de productos por debajo del mínimo (patrón Observer).

Hereda de o implementa: `IDisposable`.

Miembro	Tipo	Descripción
`Revisar`	Método	Fuerza una revisión inmediata, por ejemplo después de registrar un movimiento.

9.6.19. PedidoService

Coordina la creación, validación y persistencia de los pedidos comerciales.

Miembro	Tipo	Descripción
`ReservarExistencias(Comercial.Pedido)`	Método	Reserva las existencias de las líneas de producto del catálogo. Si alguna no alcanza, no se reserva nada.
`Cancelar(System.Guid,System.String)`	Método	Cancela un pedido que todavía no fue despachado ni facturado: libera sus reservas y cancela las órdenes de fabricación pendientes.

9.6.20. PreparacionService

Deja un pedido en condiciones de despacharse: reserva el stock que falta, ingresa la mercadería faltante si se pide y completa los materiales de una orden de fabricación.

Miembro	Tipo	Descripción
`IngresarStock(System.Guid,System.Decimal,System.String)`	Método	Ingresa existencias de un producto en su ubicación habitual como ajuste positivo.
`PrepararPedido(System.Guid,System.Boolean)`	Método	Reserva para el pedido lo que le falta. Si no hay stock suficiente y se indica, ingresa primero la cantidad faltante. Las líneas de fabricación especial no se tocan: se resuelven en Producción.
`CompletarMateriales(System.Guid,System.Boolean)`	Método	Reserva los materiales de una orden de fabricación e ingresa los que falten. Devuelve los materiales que todavía no se pudieron cubrir (vacío si está todo).
`FabricarCompleto(System.Guid)`	Método	Recorre todo el ciclo de una orden de fabricación: materiales, inicio, avance al 100%, consumo, control de calidad aprobado y entrada del producto terminado.

9.6.21. PresupuestoService

Coordina las operaciones relacionadas con presupuestos.

Miembro	Tipo	Descripción
`SenaMinimaFabricacion`	Campo	Seña mínima que exige el pedido de productos a medida.
`DescuentoMaximoSinAutorizacion`	Campo	Descuento que puede aplicar un vendedor sin autorización.
`CrearPresupuesto(DTOs.PresupuestoSolicitud)`	Método	Genera un presupuesto a partir de líneas del catálogo o de servicios, aplicando las reglas comerciales (vigencia, descuentos y seña mínima).

`ActualizarVencidos`	Método	Marca como vencidos los presupuestos abiertos cuya vigencia terminó.
`ExtenderVigencia(System.Guid, System.Int32)`	Método	Renueva la vigencia de un presupuesto vencido o abierto para poder convertirlo.

9.6.22. ProduccionService

Órdenes de fabricación: generación, reserva de materiales, avance, consumos, control de calidad y finalización.

Miembro	Tipo	Descripción
`GenerarOrdenesDelPedido(System.Guid)`	Método	Crea una orden de fabricacion por cada linea a medida del pedido que todavia no tiene una. Usa como material sugerido la materia prima con mas existencia (Produccion puede ajustarlo despues).
`ElegirMaterial(Generic.List{DTOs.ProductoListaDto},DTOs.PedidoPendienteFabricacionDto)`	Método	Sugiere la materia prima que corresponde al producto: misma calidad de acero, misma forma (chapa o cano) y mismo diametro cuando figura en la descripcion; a igualdad, la de mas existencia.
`ObtenerOrdenesPorAceptar`	Método	Ordenes creadas que todavia no se aceptaron (estado Pendiente), con el presupuesto, el cliente y el estado de pago.
`ExigirFacturaConSena(Store.IUnidadDeTrabajo,Comercial.Pedido)`	Método	Un producto a medida nunca entra a fabricacion si el cliente no abono la sena minima.
`GenerarOrdenReproceso(Store.IUnidadDeTrabajo,System.Guid,System.Decimal,System.Nullable{System.Guid},System.String)`	Método	Crea una orden de reproceso para un producto devuelto, sin pedido asociado.
`ReservarFaltantes(Store.IUnidadDeTrabajo,Produccion.OrdenFabricacion)`	Método	Reserva, para cada material, lo que haya disponible. Devuelve lo que falta.
`ObtenerResumenOrden(System.Guid)`	Método	Toda la vida de una orden: creacion, aceptacion, avances, calidad, cierre o cancelacion (con motivo y usuario).

9.6.23. ReporteService

Indicadores y reportes de gestión. Todos los importes se expresan en pesos; los pedidos en dólares se convierten con su tipo de cambio.

9.6.24. RespaldoService

Coordina el respaldo de las bases de negocio y servicios.

9.6.25. SeguimientoService

Consulta del estado de un pedido en todas sus etapas: comercial, productiva, de facturación y logística.

Miembro	Tipo	Descripción
`ObtenerLineaDeTiempo(System.Guid)`	Método	Reúne en orden cronológico todo lo que le pasó al pedido.
`SinHoraDespuesDe(System.DateTime,System.DateTime,System.Int32)`	Método	Las fechas que se cargan sin hora (0:00) se ubican justo despues del hecho que las origina, para que el historial siga el orden real de los pasos.

9.6.26. SesionActual

Mantiene los datos del usuario autenticado durante la ejecución.

Miembro	Tipo	Descripción
`NombreUsuario`	Propiedad	Nombre del usuario para registrar quién realizó una operación.

9.6.27. UsuarioService

Coordina el registro, autenticacion y administracion de los usuarios de Edelstahl ERP.

Miembro	Tipo	Descripción
`ValidarClave(System.String)`	Método	La clave se valida como regla de negocio para mostrar un aviso claro y no un error inesperado.

9.7. Espacio de nombres Edelstahl.BLL.Validators

Capa: **Lógica de negocio**. 1 tipo(s) con documentación.

Tipo	Descripción
`ClienteValidator`	Centraliza las validaciones aplicables a los clientes.

9.7.1. ClienteValidator

Centraliza las validaciones aplicables a los clientes.

9.8. Espacio de nombres Edelstahl.DAL.Backup

Capa: **Acceso a datos**. 1 tipo(s) con documentación.

Tipo	Descripción
`RespaldoDatabaseService`	Ejecuta respaldos completos de SQL Server. Esta operacion administrativa utiliza ADO.NET.

9.8.1. RespaldoDatabaseService

Ejecuta respaldos completos de SQL Server. Esta operacion administrativa utiliza ADO.NET.

9.9. Espacio de nombres Edelstahl.DAL.EntityFramework

Capa: **Acceso a datos**. 7 tipo(s) con documentación.

Tipo	Descripción
`ClienteConfiguration`	Configuración de Entity Framework para la entidad Cliente.
`EdelstahlNegocioContext`	Contexto de Entity Framework 6 para la base EdelstahlNegocio.
`EsquemaNegocio`	Crea las bases de datos y las tablas que falten, sin modificar las que ya existen ni sus datos. Las tablas se obtienen del modelo de Entity Framework, de modo que el esquema siempre coincide con las clases del dominio. Las referencias entre módulos (claves foráneas) y los índices se declaran en este archivo.
`EsquemaServicios`	Crea la base de servicios (usuarios, roles y bitácora) cuando falta. Las tablas que ya existen no se modifican.
`InicializadorNegocio`	Inicializa la conexión con la base de datos comercial administrada mediante Entity Framework 6. También inicializa y verifica los controles de integridad DVH y DVV de la tabla Clientes.
`IntegridadTablaConfiguration`	Configuración de Entity Framework para el control de integridad vertical.
`ModeloNegocioConfiguration`	Configuración de las tablas de los módulos de catálogo, inventario, producción, compras, finanzas, logística y devoluciones. Las reglas generales (longitudes y precisión decimal) se definen por convención sobre las clases de estos módulos; lo particular de cada tabla (nombre, índices y relaciones padre-hijo) se declara abajo.

9.9.1. ClienteConfiguration

Configuración de Entity Framework para la entidad Cliente.

Hereda de o implementa: `EntityTypeConfiguration`.

9.9.2. EdelstahlNegocioContext

Contexto de Entity Framework 6 para la base EdelstahlNegocio.

Hereda de o implementa: `DbContext`.

9.9.3. EsquemaNegocio

Crea las bases de datos y las tablas que falten, sin modificar las que ya existen ni sus datos. Las tablas se obtienen del modelo de Entity Framework, de modo que el esquema siempre coincide con las clases del dominio. Las referencias entre módulos (claves foráneas) y los índices se declaran en este archivo.

Miembro	Tipo	Descripción
`Referencias`	Campo	Tabla, columna y tabla referenciada.
`Indices`	Campo	Tabla, columnas separadas por coma y si es único.
`Instalar`	Método	Crea la base si no existe y agrega las tablas, referencias e índices que falten.
`GenerarScript`	Método	Devuelve el script completo (tablas, referencias e índices) para documentación o para crear la base manualmente.
`AgregarColumnasFaltantes(SqlClient.SqlConnection,SqlClient.SqlTransaction,Generic.List{System.String},Generic.HashSet{System.String})`	Método	Las tablas que ya existian (por ejemplo la de clientes de una version anterior) reciben las columnas nuevas del modelo. Las columnas obligatorias se agregan con un valor inicial para no fallar si ya hay filas.
`AsegurarBase(System.String)`	Método	Crea la base de datos indicada si todavía no existe.

9.9.4. EsquemaServicios

Crea la base de servicios (usuarios, roles y bitácora) cuando falta. Las tablas que ya existen no se modifican.

9.9.5. InicializadorNegocio

Inicializa la conexión con la base de datos comercial administrada mediante Entity Framework 6. También inicializa y verifica los controles de integridad DVH y DVV de la tabla Clientes.

Miembro	Tipo	Descripción

`MigrarCifradoClientes`	Método	Reemplaza el formato de cifrado anterior del CUIT por el actual y actualiza DVH y DVV dentro de la misma transacción. Solo actúa cuando la integridad previa es válida; si no lo es, no modifica nada y la verificación posterior informa la alteración.
`ObtenerCuitPlano(System.String)`	Método	Devuelve el CUIT legible de un valor guardado o null si no puede descifrarse con la clave actual.
`MigrarDvhClientesExistentes`	Método	Genera el DVH de los clientes creados antes de incorporar el control de integridad.
`InicializarOVerificarDvvClientes`	Método	Crea el primer DVV de Clientes cuando todavía no existe o verifica el valor ya almacenado.

9.9.6. IntegridadTablaConfiguration

Configuración de Entity Framework para el control de integridad vertical.

Hereda de o implementa: `EntityTypeConfiguration`.

9.9.7. ModeloNegocioConfiguration

Configuración de las tablas de los módulos de catálogo, inventario, producción, compras, finanzas, logística y devoluciones. Las reglas generales (longitudes y precisión decimal) se definen por convención sobre las clases de estos módulos; lo particular de cada tabla (nombre, índices y relaciones padre-hijo) se declara abajo.

9.10. Espacio de nombres Edelstahl.DAL.Factory

Capa: **Acceso a datos**. 1 tipo(s) con documentación.

Tipo	Descripción
`FactoryDataAccess`	Centraliza los repositorios utilizados por Edelstahl ERP. En modo SQL Server utiliza Entity Framework y ADO.NET. En modo demostración utiliza repositorios en memoria.

9.10.1. FactoryDataAccess

Centraliza los repositorios utilizados por Edelstahl ERP. En modo SQL Server utiliza Entity Framework y ADO.NET. En modo demostración utiliza repositorios en memoria.

Miembro	Tipo	Descripción
`NegocioStore`	Propiedad	Almacén transaccional de los módulos de catálogo, inventario, producción, compras, finanzas y logística.

`Inicializar(Configuration.ExecutionMode)`	Método	Inicializa los repositorios según el modo de ejecución seleccionado al comenzar la aplicación.
`InicializarDemo`	Método	Configura los repositorios temporales en memoria. Los datos se conservan mientras la aplicación permanece abierta.

9.11. Espacio de nombres Edelstahl.DAL.Implementations.EntityFramework

Capa: **Acceso a datos**. 3 tipo(s) con documentación.

Tipo	Descripción
`ClienteRepositoryEntityFramework`	Persistencia de clientes mediante Entity Framework 6. El CUIT se cifra mediante AES antes de almacenarse. El DVH protege la integridad de cada cliente. El DVV protege la integridad del conjunto completo.
`PedidoRepositoryEntityFramework`	Persistencia de pedidos y detalles mediante Entity Framework 6.
`PresupuestoRepositoryEntityFramework`	Persistencia de presupuestos y detalles mediante Entity Framework 6.

9.11.1. ClienteRepositoryEntityFramework

Persistencia de clientes mediante Entity Framework 6. El CUIT se cifra mediante AES antes de almacenarse. El DVH protege la integridad de cada cliente. El DVV protege la integridad del conjunto completo.

Hereda de o implementa: `IClienteRepository`.

Miembro	Tipo	Descripción
`GuardarClientesYActualizarDvv(EntityFramework.EdelstahlNegocioContext)`	Método	Guarda las modificaciones legítimas sobre clientes y actualiza el DVV dentro de la misma transacción.
`VerificarDvvActual(EntityFramework.EdelstahlNegocioContext)`	Método	Comprueba que el conjunto de clientes no haya sido alterado fuera de Edelstahl ERP.

9.11.2. PedidoRepositoryEntityFramework

Persistencia de pedidos y detalles mediante Entity Framework 6.

Hereda de o implementa: `IPedidoRepository`.

9.11.3. PresupuestoRepositoryEntityFramework

Persistencia de presupuestos y detalles mediante Entity Framework 6.

Hereda de o implementa: `IPresupuestoRepository`.

9.12. Espacio de nombres Edelstahl.DAL.Implementations.Memory

Capa: **Acceso a datos**. 4 tipo(s) con documentación.

Tipo	Descripción
`BitacoraRepositoryMemory`	Bitácora en memoria para el modo demostración. El acceso está protegido porque se escribe desde varios hilos.
`ClienteRepositoryMemory`	Implementación temporal del repositorio de clientes. Mantiene la información en memoria mientras la aplicación está abierta.
`PedidoRepositoryMemory`	Implementación en memoria del repositorio de pedidos.
`PresupuestoRepositoryMemory`	Implementación temporal del repositorio de presupuestos. Los datos se conservan mientras la aplicación está abierta.

9.12.1. BitacoraRepositoryMemory

Bitácora en memoria para el modo demostración. El acceso está protegido porque se escribe desde varios hilos.

Hereda de o implementa: `IBitacoraRepository`.

9.12.2. ClienteRepositoryMemory

Implementación temporal del repositorio de clientes. Mantiene la información en memoria mientras la aplicación está abierta.

Hereda de o implementa: `IClienteRepository`.

9.12.3. PedidoRepositoryMemory

Implementación en memoria del repositorio de pedidos.

Hereda de o implementa: `IPedidoRepository`.

9.12.4. PresupuestoRepositoryMemory

Implementación temporal del repositorio de presupuestos. Los datos se conservan mientras la aplicación está abierta.

Hereda de o implementa: `IPresupuestoRepository`.

9.13. Espacio de nombres Edelstahl.DAL.Implementations.SqlServer

Capa: **Acceso a datos**. 2 tipo(s) con documentación.

Tipo	Descripción
`RolRepositorySqlServer`	Guarda y recupera roles desde SQL Server.
`UsuarioRepositorySqlServer`	Guarda y recupera usuarios y su rol desde SQL Server.

9.13.1. RolRepositorySqlServer

Guarda y recupera roles desde SQL Server.

Hereda de o implementa: `IRolRepository`.

9.13.2. UsuarioRepositorySqlServer

Guarda y recupera usuarios y su rol desde SQL Server.

Hereda de o implementa: `IUsuarioRepository`.

9.14. Espacio de nombres Edelstahl.DAL.Interfaces

Capa: **Acceso a datos**. 6 tipo(s) con documentación.

Tipo	Descripción
`IClienteRepository`	Define las operaciones de persistencia específicas para la entidad Cliente.
`IGenericRepository`	Define las operaciones básicas de persistencia para las entidades del sistema.
`IPedidoRepository`	Define las operaciones de persistencia relacionadas con los pedidos.
`IPresupuestoRepository`	Define las operaciones de persistencia relacionadas con los presupuestos.
`IRolRepository`	Define las operaciones de persistencia relacionadas con roles.
`IUsuarioRepository`	Define las operaciones de persistencia relacionadas con usuarios.

9.14.1. IClienteRepository

Define las operaciones de persistencia específicas para la entidad Cliente.

Hereda de o implementa: `IGenericRepository`.

9.14.2. IGenericRepository

Define las operaciones básicas de persistencia para las entidades del sistema.

9.14.3. IPedidoRepository

Define las operaciones de persistencia relacionadas con los pedidos.

Hereda de o implementa: `IGenericRepository`.

Miembro	Tipo	Descripción
`ActualizarEstado(System.Guid, Comercial.EstadoPedido)`	Método	Cambia solo el estado del pedido, sin reescribir sus líneas.

9.14.4. IPresupuestoRepository

Define las operaciones de persistencia relacionadas con los presupuestos.

Hereda de o implementa: `IGenericRepository`.

9.14.5. IRolRepository

Define las operaciones de persistencia relacionadas con roles.

Hereda de o implementa: `IGenericRepository`.

9.14.6. IUsuarioRepository

Define las operaciones de persistencia relacionadas con usuarios.

Hereda de o implementa: `IGenericRepository`.

9.15. Espacio de nombres Edelstahl.DAL.Store

Capa: **Acceso a datos**. 5 tipo(s) con documentación.

Tipo	Descripción
`EfNegocioStore`	Almacén que persiste en SQL Server mediante Entity Framework. Cada ejecución usa su propio contexto y una única transacción.
`IConjunto`	Conjunto de entidades de un tipo dentro de una unidad de trabajo. Se consulta con LINQ y se modifica mediante Agregar y Quitar.
`INegocioStore`	Punto único de acceso a los datos de los módulos de negocio. La implementación depende del modo de ejecución.
`IUnidadDeTrabajo`	Agrupa las operaciones que deben confirmarse o revertirse juntas.
`MemoryNegocioStore`	Almacén temporal usado en el modo demostración. Los datos viven mientras la aplicación está abierta. El acceso se serializa con un candado, de modo que varios hilos (por ejemplo el monitor de stock) no vean estados intermedios. Si una operación falla, las altas y bajas de registros se revierten.

9.15.1. EfNegocioStore

Almacén que persiste en SQL Server mediante Entity Framework. Cada ejecución usa su propio contexto y una única transacción.

Hereda de o implementa: `INegocioStore`.

9.15.2. IConjunto

Conjunto de entidades de un tipo dentro de una unidad de trabajo. Se consulta con LINQ y se modifica mediante Agregar y Quitar.

Hereda de o implementa: `IQueryable where T : Entity`.

Miembro	Tipo	Descripción
`Incluir(System.String)`	Método	Carga una propiedad de navegación (Detalles, Materiales, etc.). En el almacén en memoria las colecciones ya están cargadas.

9.15.3. INegocioStore

Punto único de acceso a los datos de los módulos de negocio. La implementación depende del modo de ejecución.

9.15.4. IUnidadDeTrabajo

Agrupa las operaciones que deben confirmarse o revertirse juntas.

Miembro	Tipo	Descripción
`Guardar`	Método	Envía a la base los cambios pendientes sin confirmar la transacción. Se usa cuando una entidad debe existir antes de ser referenciada.

9.15.5. MemoryNegocioStore

Almacén temporal usado en el modo demostración. Los datos viven mientras la aplicación está abierta. El acceso se serializa con un candado, de modo que varios hilos (por ejemplo el monitor de stock) no vean estados intermedios. Si una operación falla, las altas y bajas de registros se revierten.

Hereda de o implementa: `INegocioStore`.

Miembro	Tipo	Descripción
`RegistrarHijos(System.Object)`	Método	Registra, como conjuntos propios, los registros hijos que cuelgan de una entidad (detalles, ubicaciones, materiales). Así se pueden consultar igual que en la base de datos.

`TomarCopia`	Método	Guarda la composición de cada lista para poder revertir el alta o la baja de entidades si la operación falla.

9.16. Espacio de nombres Edelstahl.DAL.Tools

Capa: **Acceso a datos**. 1 tipo(s) con documentación.

Tipo	Descripción
`SqlServerConnection`	Centraliza las conexiones con las bases de datos de Edelstahl ERP. La instancia de SQL Server se detecta sola (SQL Server Express o LocalDB) y puede fijarse en App.config con las claves SqlServerInstancia, BaseNegocio y BaseServicios. También admiten variables de entorno (EDELSTAHL_SQL_SERVER, EDELSTAHL_NEGOCIO_DB, EDELSTAHL_SERVICIOS_DB), útiles para ejecutar pruebas sobre bases separadas.

9.16.1. SqlServerConnection

Centraliza las conexiones con las bases de datos de Edelstahl ERP. La instancia de SQL Server se detecta sola (SQL Server Express o LocalDB) y puede fijarse en App.config con las claves SqlServerInstancia, BaseNegocio y BaseServicios. También admiten variables de entorno (EDELSTAHL_SQL_SERVER, EDELSTAHL_NEGOCIO_DB, EDELSTAHL_SERVICIOS_DB), útiles para ejecutar pruebas sobre bases separadas.

Miembro	Tipo	Descripción
`Reiniciar`	Método	Descarta la instancia detectada para volver a buscarla.

9.17. Espacio de nombres Edelstahl.Domain.Catalogo

Capa: **Dominio**. 5 tipo(s) con documentación.

Tipo	Descripción
`CategoriaProducto`	Categoría del catálogo. Admite jerarquía mediante el identificador de la categoría padre.
`MovimientoStock`	Es el registro de cada entrada, salida o ajuste de mercadería: qué producto, en qué ubicación, cuánto, quién lo hizo y con qué documento de origen. Con esto queda la historia completa del stock.
`Producto`	Es cada artículo que maneja la empresa: chapas, caños, barras y otros productos de acero inoxidable. Guarda código SKU, nombre, categoría, unidad de medida, calidad AISI, espesor, acabado, medidas, precio y puntos de reposición. Un producto puede venderse, comprarse y/o fabricarse.

`StockProducto`	Es la existencia de un producto en una ubicación del depósito. Guarda la cantidad física, la reservada para pedidos, la bloqueada y el costo promedio. El disponible real es lo físico menos lo reservado y lo bloqueado.
`TipoProducto`	Clasificación funcional del producto: determina si puede comprarse, fabricarse o venderse.

9.17.1. CategoriaProducto

Categoría del catálogo. Admite jerarquía mediante el identificador de la categoría padre.

Hereda de o implementa: `Entity`.

9.17.2. MovimientoStock

Hereda de o implementa: `Entity`.

Miembro	Tipo	Descripción
`Incrementa`	Propiedad	Indica si el movimiento incrementa la existencia física.

9.17.3. Producto

Hereda de o implementa: `Entity`.

Miembro	Tipo	Descripción
`AjustarSegunTipo`	Método	Aplica los valores por omisión que corresponden al tipo.

9.17.4. StockProducto

Hereda de o implementa: `Entity`.

Miembro	Tipo	Descripción
`CantidadDisponible`	Propiedad	Existencia libre: física menos reservada y bloqueada.

9.17.5. TipoProducto

Clasificación funcional del producto: determina si puede comprarse, fabricarse o venderse.

9.18. Espacio de nombres Edelstahl.Domain.Comercial

Capa: **Dominio**. 9 tipo(s) con documentación.

Tipo	Descripción
`Cliente`	Representa a un cliente de Edelstahl.
`DetallePedido`	Representa un producto, servicio o fabricación especial incluido en un pedido confirmado.
`DetallePresupuesto`	Representa un renglón o posición dentro de un presupuesto.
`EstadoPresupuesto`	Representa los posibles estados comerciales de un presupuesto.
`Moneda`	Monedas admitidas en presupuestos y pedidos.
`Pedido`	Representa un pedido comercial confirmado a partir de un presupuesto aceptado.
`Presupuesto`	Representa una cotización comercial emitida para un cliente. Puede contener productos, servicios y fabricaciones especiales.
`TipoItemPresupuesto`	Clasifica los diferentes conceptos que pueden incluirse en una cotización.
`TipoPersona`	Una empresa se identifica con CUIT; un particular, con CUIL.

9.18.1. Cliente

Representa a un cliente de Edelstahl.

Hereda de o implementa: `Entity`.

Miembro	Tipo	Descripción
`Tipo`	Propiedad	Empresa (CUIT) o particular (CUIL). El número se guarda siempre en CUIT.
`EtiquetaDocumento`	Propiedad	Texto del documento según el tipo de cliente: CUIT o CUIL.
`AutorizadoACredito`	Propiedad	Cliente autorizado a recibir la mercaderia sin haber pagado el 100% (cuenta corriente).
`DVH`	Propiedad	Dígito Verificador Horizontal del registro. Contiene un hash SHA-256 representado mediante 64 caracteres hexadecimales.

9.18.2. DetallePedido

Representa un producto, servicio o fabricación especial incluido en un pedido confirmado.

Hereda de o implementa: `Entity`.

9.18.3. DetallePresupuesto

Representa un renglón o posición dentro de un presupuesto.

Hereda de o implementa: `Entity`.

9.18.4. EstadoPresupuesto

Representa los posibles estados comerciales de un presupuesto.

9.18.5. Moneda

Monedas admitidas en presupuestos y pedidos.

9.18.6. Pedido

Representa un pedido comercial confirmado a partir de un presupuesto aceptado.

Hereda de o implementa: `Entity`.

9.18.7. Presupuesto

Representa una cotización comercial emitida para un cliente. Puede contener productos, servicios y fabricaciones especiales.

Hereda de o implementa: `Entity`.

Miembro	Tipo	Descripción
`AnticipoMinimoPorcentaje`	Campo	Porcentaje minimo de sena que exige toda venta antes de convertirse en pedido.
`PorcentajeAnticipoEfectivo`	Método	Porcentaje de sena que se aplica: el del presupuesto y, si incluye productos a fabricar, nunca menos que el minimo. Los productos de stock no exigen sena (se pagan al 100% antes de remitir).

9.18.8. TipoItemPresupuesto

Clasifica los diferentes conceptos que pueden incluirse en una cotización.

9.18.9. TipoPersona

Una empresa se identifica con CUIT; un particular, con CUIL.

9.19. Espacio de nombres Edelstahl.Domain.Common

Capa: **Dominio**. 1 tipo(s) con documentación.

Tipo	Descripción
`Entity`	Clase base para las entidades persistentes del sistema Edelstahl.

9.19.1. Entity

Clase base para las entidades persistentes del sistema Edelstahl.

Miembro	Tipo	Descripción
`Id`	Propiedad	Identificador único de la entidad.

9.20. Espacio de nombres Edelstahl.Domain.Finanzas

Capa: **Dominio**. 1 tipo(s) con documentación.

Tipo	Descripción
`TipoComprobante`	Tipo de comprobante fiscal emitido. La factura A discrimina el IVA; la B lo incluye dentro del precio final.

9.20.1. TipoComprobante

Tipo de comprobante fiscal emitido. La factura A discrimina el IVA; la B lo incluye dentro del precio final.

9.21. Espacio de nombres Edelstahl.Domain.Integrity

Capa: **Dominio**. 1 tipo(s) con documentación.

Tipo	Descripción
`IntegridadTabla`	Almacena el Dígito Verificador Vertical correspondiente a una tabla controlada.

9.21.1. IntegridadTabla

Almacena el Dígito Verificador Vertical correspondiente a una tabla controlada.

Miembro	Tipo	Descripción
`NombreTabla`	Propiedad	Nombre lógico de la tabla protegida. Funciona como clave primaria.
`DVV`	Propiedad	Hash SHA-256 calculado a partir de los DVH de los registros de la tabla.
`FechaActualizacion`	Propiedad	Fecha de la última actualización válida.

9.22. Espacio de nombres Edelstahl.Domain.Security

Capa: **Dominio**. 5 tipo(s) con documentación.

Tipo	Descripción
`MatrizPermisos`	Matriz de permisos por rol (control de acceso basado en roles).
`NombresRol`	Nombres de los roles del sistema.
`Permiso`	Acciones que se pueden autorizar. Cada rol recibe un subconjunto.
`Rol`	Representa un perfil de acceso dentro de Edelstahl ERP.
`Usuario`	Representa un usuario autorizado para utilizar Edelstahl ERP. La contraseña nunca se almacena en texto plano.

9.22.1. MatrizPermisos

Matriz de permisos por rol (control de acceso basado en roles).

9.22.2. NombresRol

Nombres de los roles del sistema.

9.22.3. Permiso

Acciones que se pueden autorizar. Cada rol recibe un subconjunto.

9.22.4. Rol

Representa un perfil de acceso dentro de Edelstahl ERP.

Hereda de o implementa: `Entity`.

9.22.5. Usuario

Representa un usuario autorizado para utilizar Edelstahl ERP. La contraseña nunca se almacena en texto plano.

Hereda de o implementa: `Entity`.

9.23. Espacio de nombres Edelstahl.Services.Configuration

Capa: **Servicios transversales**. 2 tipo(s) con documentación.

Tipo	Descripción
`ApplicationMode`	Mantiene el modo de ejecución seleccionado durante la sesión actual de Edelstahl ERP.

`ExecutionMode`	Define los modos disponibles para iniciar Edelstahl ERP.

9.23.1. ApplicationMode

Mantiene el modo de ejecución seleccionado durante la sesión actual de Edelstahl ERP.

9.23.2. ExecutionMode

Define los modos disponibles para iniciar Edelstahl ERP.

9.24. Espacio de nombres Edelstahl.Services.Cryptography

Capa: **Servicios transversales**. 3 tipo(s) con documentación.

Tipo	Descripción
`AesEncryptionService`	Proporciona cifrado simétrico AES-256 (CBC) con autenticación HMAC-SHA256 para proteger campos sensibles. Formato actual ("v2:"): la clave maestra se deriva una sola vez por proceso con PBKDF2 y cada mensaje obtiene sus claves con una sal propia mediante HMAC. Así cifrar o descifrar cuesta microsegundos. Formato anterior (sin prefijo): cada mensaje derivaba sus claves con PBKDF2 completo, lo que tardaba cerca de un segundo. Se sigue pudiendo leer para no invalidar los datos ya guardados.
`CryptographyService`	Proporciona una fachada para las operaciones criptográficas utilizadas por Edelstahl.
`CryptographySettings`	Obtiene la configuración utilizada por los servicios criptográficos de Edelstahl.

9.24.1. AesEncryptionService

Proporciona cifrado simétrico AES-256 (CBC) con autenticación HMAC-SHA256 para proteger campos sensibles. Formato actual ("v2:"): la clave maestra se deriva una sola vez por proceso con PBKDF2 y cada mensaje obtiene sus claves con una sal propia mediante HMAC. Así cifrar o descifrar cuesta microsegundos. Formato anterior (sin prefijo): cada mensaje derivaba sus claves con PBKDF2 completo, lo que tardaba cerca de un segundo. Se sigue pudiendo leer para no invalidar los datos ya guardados.

Miembro	Tipo	Descripción
`Precalentar`	Método	Calcula por adelantado la clave maestra para que la primera operación de cifrado no demore. Conviene llamarlo en segundo plano.
`Encrypt(System.String)`	Método	Cifra un texto con el formato actual.

`EncryptLegacy(System.String)`	Método	Cifra con el formato anterior. Solo se conserva para pruebas de compatibilidad con datos antiguos.
`Decrypt(System.String)`	Método	Descifra un texto generado por Encrypt (formato actual o anterior).

9.24.2. CryptographyService

Proporciona una fachada para las operaciones criptográficas utilizadas por Edelstahl.

Miembro	Tipo	Descripción
`Encrypt(System.String)`	Método	Cifra un texto mediante el servicio AES configurado para la aplicación.
`Decrypt(System.String)`	Método	Descifra un texto previamente generado mediante el servicio AES.
`Precalentar`	Método	Calcula por adelantado la clave maestra. Se invoca en segundo plano al iniciar para que el primer cifrado no demore.

9.24.3. CryptographySettings

Obtiene la configuración utilizada por los servicios criptográficos de Edelstahl.

Miembro	Tipo	Descripción
`GetEncryptionSecret`	Método	Obtiene la clave configurada para el cifrado simétrico AES.

9.25. Espacio de nombres Edelstahl.Services.Documents

Capa: **Servicios transversales**. 10 tipo(s) con documentación.

Tipo	Descripción
`ComprobantePedidoPdfService`	Genera el comprobante PDF de un pedido confirmado.
`DatosEmisor`	Datos de la empresa que emite los documentos. Son ficticios y se pueden cambiar en App.config (claves Emisor...).
`DatosFactura`	Factura o nota de crédito. La letra A discrimina el IVA; en la B el IVA está incluido en los precios.
`DocumentoPdfBase`	Patrón Template Method: esta clase fija el esqueleto de todos los documentos (página, encabezado, cuerpo, pie y guardado) y

	cada documento concreto completa solo el encabezado y el cuerpo.
`FacturaPdfService`	Factura A, factura B y nota de crédito. En la A se discrimina el IVA; en la B los precios ya lo incluyen.
`FormatoDocumento`	Formatos comunes a todos los documentos.
`OrdenCompraPdfService`	Orden de compra dirigida a un proveedor.
`PresupuestoPdfService`	Genera el presupuesto en PDF para que el cliente se lo lleve y lo evalúe con calma. Indica hasta cuándo es válido y que los precios pueden variar pasada esa fecha.
`RemitoPdfService`	Remito de entrega en tres copias (original, duplicado y triplicado), cada una impresa sobre una hoja de color pastel: verde, amarillo y rosa.
`VarianteDocumento`	Una copia del documento: su etiqueta (Original, Duplicado...) y el color de fondo de la hoja.

9.25.1. ComprobantePedidoPdfService

Genera el comprobante PDF de un pedido confirmado.

9.25.2. DatosEmisor

Datos de la empresa que emite los documentos. Son ficticios y se pueden cambiar en App.config (claves Emisor...).

9.25.3. DatosFactura

Factura o nota de crédito. La letra A discrimina el IVA; en la B el IVA está incluido en los precios.

9.25.4. DocumentoPdfBase

Patrón Template Method: esta clase fija el esqueleto de todos los documentos (página, encabezado, cuerpo, pie y guardado) y cada documento concreto completa solo el encabezado y el cuerpo.

Miembro	Tipo	Descripción
`Variantes`	Método	Por omisión el documento tiene una sola copia sobre fondo blanco.
`Generar`	Método	Genera el PDF y devuelve la ruta del archivo.

`GenerarImagenes(System.String)`	Método	Exporta cada pagina como imagen PNG (sirve para revisar el diseno y para las pruebas).
`Politica`	Propiedad	Texto opcional de politica que se imprime en el pie (facturas y remitos).

9.25.5. FacturaPdfService

Factura A, factura B y nota de crédito. En la A se discrimina el IVA; en la B los precios ya lo incluyen.

Hereda de o implementa: `DocumentoPdfBase`.

9.25.6. FormatoDocumento

Formatos comunes a todos los documentos.

Miembro	Tipo	Descripción
`Logo`	Método	Logo de Edelstahl (Resources/Logo/logo_edelstahl.png) o null si no esta.
`PoliticaDeEntrega`	Método	Texto de la politica de entrega y devoluciones que se imprime en facturas y remitos.

9.25.7. OrdenCompraPdfService

Orden de compra dirigida a un proveedor.

Hereda de o implementa: `DocumentoPdfBase`.

9.25.8. PresupuestoPdfService

Genera el presupuesto en PDF para que el cliente se lo lleve y lo evalúe con calma. Indica hasta cuándo es válido y que los precios pueden variar pasada esa fecha.

Miembro	Tipo	Descripción
`GenerarImagenes(Comercial.Presupuesto,Comercial.Cliente,System.String)`	Método	Genera el presupuesto como imagenes (una por pagina), para revisarlo sin abrir el PDF.

9.25.9. RemitoPdfService

Remito de entrega en tres copias (original, duplicado y triplicado), cada una impresa sobre una hoja de color pastel: verde, amarillo y rosa.

Hereda de o implementa: `DocumentoPdfBase`.

9.25.10. VarianteDocumento

Una copia del documento: su etiqueta (Original, Duplicado...) y el color de fondo de la hoja.

9.26. Espacio de nombres Edelstahl.Services.Export

Capa: **Servicios transversales**. 3 tipo(s) con documentación.

Tipo	Descripción
`ExportadorBase`	Patrón Template Method: el esqueleto de la exportación (encabezado, filas y escritura del archivo) es único; cada exportador solo define cómo se arman el encabezado y cada fila.
`ExportadorCsv`	Exporta a CSV cualquier lista de objetos. Usa reflexión para leer las propiedades de la clase, por eso sirve para tipos que todavía no existen.
`SerializadorXml`	Serialización XML de listas de datos (exportar y volver a leer).

9.26.1. ExportadorBase

Patrón Template Method: el esqueleto de la exportación (encabezado, filas y escritura del archivo) es único; cada exportador solo define cómo se arman el encabezado y cada fila.

9.26.2. ExportadorCsv

Exporta a CSV cualquier lista de objetos. Usa reflexión para leer las propiedades de la clase, por eso sirve para tipos que todavía no existen.

Hereda de o implementa: `ExportadorBase`.

9.26.3. SerializadorXml

Serialización XML de listas de datos (exportar y volver a leer).

9.27. Espacio de nombres Edelstahl.Services.Integrity

Capa: **Servicios transversales**. 2 tipo(s) con documentación.

Tipo	Descripción
`ClienteDvhService`	Calcula y verifica el Dígito Verificador Horizontal de un cliente. El DVH se obtiene mediante SHA-256 sobre una representación determinística de los campos protegidos del registro.
`ClienteDvvService`	Calcula y verifica el Dígito Verificador Vertical correspondiente a la tabla Clientes. El DVV se genera mediante SHA-256 a partir del Id y el DVH de todos los clientes, ordenados de forma determinística.

9.27.1. ClienteDvhService

Calcula y verifica el Dígito Verificador Horizontal de un cliente. El DVH se obtiene mediante SHA-256 sobre una representación determinística de los campos protegidos del registro.

Miembro	Tipo	Descripción
`Calcular(Comercial.Cliente)`	Método	Calcula el DVH correspondiente al estado actual del cliente. Cuando se utiliza con Entity Framework, el CUIT debe contener exactamente el valor que será almacenado en la base de datos.
`EsValido(Comercial.Cliente)`	Método	Comprueba que el DVH almacenado coincida con el valor calculado actualmente.
`Asignar(Comercial.Cliente)`	Método	Calcula y asigna el DVH al cliente.

9.27.2. ClienteDvvService

Calcula y verifica el Dígito Verificador Vertical correspondiente a la tabla Clientes. El DVV se genera mediante SHA-256 a partir del Id y el DVH de todos los clientes, ordenados de forma determinística.

Miembro	Tipo	Descripción
`Calcular(Generic.IEnumerable{Comercial.Cliente})`	Método	Calcula el DVV del conjunto completo de clientes.
`EsValido(Generic.IEnumerable{Comercial.Cliente},System.String)`	Método	Compara el DVV almacenado con el DVV calculado para los clientes actuales.

9.28. Espacio de nombres Edelstahl.Services.Localization

Capa: **Servicios transversales**. 1 tipo(s) con documentación.

Tipo	Descripción
`LanguageService`	Administra el idioma activo y las traducciones de Edelstahl ERP.

9.28.1. LanguageService

Administra el idioma activo y las traducciones de Edelstahl ERP.

Miembro	Tipo	Descripción
`AplicarIdiomaPreferido`	Método	Aplica el idioma elegido por el usuario en una sesion anterior. Se llama despues de iniciar sesion.

9.29. Espacio de nombres Edelstahl.Services.Security

Capa: **Servicios transversales**. 2 tipo(s) con documentación.

Tipo	Descripción
`PasswordHashData`	Resultado seguro del procesamiento de una contraseña. Solo almacena el hash, la sal y la cantidad de iteraciones.
`PasswordHasher`	Genera y verifica hashes de contraseñas mediante PBKDF2. Las contraseñas nunca se guardan ni se pueden recuperar.

9.29.1. PasswordHashData

Resultado seguro del procesamiento de una contraseña. Solo almacena el hash, la sal y la cantidad de iteraciones.

9.29.2. PasswordHasher

Genera y verifica hashes de contraseñas mediante PBKDF2. Las contraseñas nunca se guardan ni se pueden recuperar.

9.30. Espacio de nombres Edelstahl.WinApp

Capa: **Interfaz**. 3 tipo(s) con documentación.

Tipo	Descripción
`FrmPrincipalV2`	Ventana principal: menu lateral por modulos (segun permisos), idioma y aviso de stock.
`Program`	Punto de entrada de la aplicación: elige el modo de ejecución, verifica la integridad de los datos, hace el respaldo automático del día y abre el inicio de sesión.
`VisorBitacoraPagina`	Pagina completa de la bitacora (acceso directo desde el menu principal).

9.30.1. FrmPrincipalV2

Ventana principal: menu lateral por modulos (segun permisos), idioma y aviso de stock.

Miembro	Tipo	Descripción
`components`	Campo	Required designer variable.
`Dispose(System.Boolean)`	Método	Clean up any resources being used. *Parámetros:* `disposing`: true if managed resources should be disposed; otherwise, false.

`InitializeComponent`	Método	Required method for Designer support - do not modify the contents of this method with the code editor.

9.30.2. Program

Miembro	Tipo	Descripción
`Main`	Método	Punto de entrada principal de Edelstahl ERP.
`OfrecerDatosDeEjemplo`	Método	Si el catalogo esta vacio carga los datos de ejemplo: automaticamente en modo demostracion y con confirmacion cuando se usa SQL Server.

9.30.3. VisorBitacoraPagina

Pagina completa de la bitacora (acceso directo desde el menu principal).

9.31. Espacio de nombres Edelstahl.WinApp.Infrastructure

Capa: **Interfaz**. 1 tipo(s) con documentación.

Tipo	Descripción
`GlobalExceptionHandler`	Captura excepciones no controladas, registra su detalle tecnico en la bitacora y muestra mensajes seguros al usuario.

9.31.1. GlobalExceptionHandler

Captura excepciones no controladas, registra su detalle tecnico en la bitacora y muestra mensajes seguros al usuario.

9.32. Espacio de nombres Edelstahl.WinApp.Modulos

Capa: **Interfaz**. 9 tipo(s) con documentación.

Tipo	Descripción
`ModuloComercial`	Ventas: clientes, presupuestos y pedidos con su seguimiento.
`ModuloFinanzas`	Finanzas: facturas, cobros, pagos a proveedores, notas de credito y cuentas a cobrar y pagar.
`ModuloInventario`	Compras y stock: catalogo, existencias, movimientos, reservas, proveedores y ordenes de compra.

`ModuloLogistica`	Logistica: remitos (original y dos copias de colores), hojas de ruta, flota y devoluciones.
`ModuloProduccion`	Produccion: lineas pendientes de fabricar, tablero de ordenes y control de calidad.
`ModuloReportes`	Reportes de gestion con rango de fechas y exportacion.
`PanelDatos`	Respaldo de bases, carga de datos de ejemplo e importacion de archivos. Las operaciones largas se ejecutan en un hilo y muestran progreso.
`PanelHojasRuta`	Hojas de ruta a la izquierda y mapa del recorrido a la derecha.
`VisorBitacora`	Pantalla de consulta de la bitacora con filtros por fecha, usuario, modulo y texto.

9.32.1. ModuloComercial

Ventas: clientes, presupuestos y pedidos con su seguimiento.

9.32.2. ModuloFinanzas

Finanzas: facturas, cobros, pagos a proveedores, notas de credito y cuentas a cobrar y pagar.

9.32.3. ModuloInventario

Compras y stock: catalogo, existencias, movimientos, reservas, proveedores y ordenes de compra.

9.32.4. ModuloLogistica

Logistica: remitos (original y dos copias de colores), hojas de ruta, flota y devoluciones.

9.32.5. ModuloProduccion

Produccion: lineas pendientes de fabricar, tablero de ordenes y control de calidad.

Miembro	Tipo	Descripción
`OfrecerFactura(System.Guid)`	Método	Al terminar de fabricar, si el pedido todavia no tiene factura se ofrece emitirla en el momento para que Logistica pueda generar el remito.

9.32.6. ModuloReportes

Reportes de gestion con rango de fechas y exportacion.

9.32.7. PanelDatos

Respaldo de bases, carga de datos de ejemplo e importacion de archivos. Las operaciones largas se ejecutan en un hilo y muestran progreso.

9.32.8. PanelHojasRuta

Hojas de ruta a la izquierda y mapa del recorrido a la derecha.

9.32.9. VisorBitacora

Pantalla de consulta de la bitacora con filtros por fecha, usuario, modulo y texto.

9.33. Espacio de nombres Edelstahl.WinApp.Properties

Capa: **Interfaz**. 1 tipo(s) con documentación.

Tipo	Descripción
`Resources`	Clase de recurso fuertemente tipado para buscar cadenas traducidas, etc.

9.33.1. Resources

Clase de recurso fuertemente tipado para buscar cadenas traducidas, etc.

Miembro	Tipo	Descripción
`ResourceManager`	Propiedad	Devuelve la instancia ResourceManager almacenada en caché utilizada por esta clase.
`Culture`	Propiedad	Invalida la propiedad CurrentUICulture del subproceso actual para todas las búsquedas de recursos usando esta clase de recursos fuertemente tipados.

9.34. Espacio de nombres Edelstahl.WinApp.Ui

Capa: **Interfaz**. 25 tipo(s) con documentación.

Tipo	Descripción
`BotonEstilo`	Boton plano que, deshabilitado, se ve claro y con texto legible (el de Windows pinta el texto gris oscuro sobre el color de fondo).
`Campo`	Describe un dato que se pide al usuario en un cuadro de dialogo.
`Dialogo`	Cuadro de dialogo generico armado a partir de una lista de campos.

`DialogoCobro`	Registra un cobro de factura. Con efectivo no se pide banco ni referencia: esos datos se piden solo si el medio de pago los requiere (cheque, transferencia, tarjeta).
`EditorLineas`	Formulario para armar las lineas de un presupuesto o de una orden de compra.
`Estilo`	Colores, fuentes y utilidades comunes de la interfaz.
`FilaCarga`	Fila de la verificacion de carga: lo que se factura y se envia, y si ya se controlo en el camion.
`FilaFaltante`	Fila editable de la ventana de faltantes: lo que falta y la cantidad a ingresar.
`FrmEspera`	Ventana modal con barra de progreso que ejecuta un trabajo en un hilo aparte.
`FrmFaltantes`	Lista en rojo de los productos que faltan, con un contador por fila para ingresar el stock. Se usa al confirmar un pedido: se agrega lo que falta y se sigue sin salir de la pantalla.
`FrmVerificarCarga`	Muestra lo que va a salir en el remito y obliga a verificarlo linea por linea al cargar el camion. Las cantidades vienen de lo facturado y reservado: no se pueden modificar.
`GeoFicticio`	Coordenadas ficticias (con la sede en Capital) con las que se dibuja el recorrido de las hojas de ruta en el mapa.
`GrillaMarca`	Grilla de datos que, si sobra espacio debajo de las filas, muestra el emblema como marca de agua.
`Lugar`	Ubicacion aproximada de localidades argentinas para dibujar el mapa de recorrido. Es un mapa esquematico y ficticio: no usa servicios externos ni claves de API.
`MapaRuta`	Mapa esquematico e interactivo del recorrido: sede en la Capital y paradas numeradas. Rueda del mouse para acercar, arrastrar para mover, doble clic para ajustar la vista.
`MapaWeb`	Mapa interactivo de la hoja de ruta: Leaflet con los mapas de OpenStreetMap dentro de WebView2. No necesita clave de API. Si el equipo no tiene WebView2 o no hay internet, muestra el mapa esquematico propio (MapaRuta).
`Marca`	Marca visual de Edelstahl: emblema (hexagono de acero con la E), textura de acero cepillado y paneles decorativos. Todo se dibuja por codigo: no usa fotos ni archivos externos.
`ModuloBase`	Modulo de la aplicacion: un conjunto de pestanas, cada una con su lista. Una pestana solo aparece si el usuario tiene el permiso requerido.

`Nombres`	Titulos en ingles de las columnas y de los permisos. El texto en ingles es la clave que luego se traduce con los archivos de idioma.
`Opcion`	Elemento de lista desplegable con texto visible y valor asociado.
`PanelAcero`	Panel de acero cepillado con el emblema; se usa en la parte libre de la barra lateral.
`PanelEncabezado`	Encabezado de la ventana: acero claro con una linea de acento.
`PanelLista`	Lista con barra de acciones, busqueda, detalle opcional y exportacion. Los datos se leen en un hilo aparte para no congelar la ventana.
`Respuesta`	Valores devueltos por el cuadro de dialogo.
`Tr`	Acceso corto a las traducciones. La clave es el texto en ingles.

9.34.1. BotonEstilo

Boton plano que, deshabilitado, se ve claro y con texto legible (el de Windows pinta el texto gris oscuro sobre el color de fondo).

9.34.2. Campo

Describe un dato que se pide al usuario en un cuadro de dialogo.

Miembro	Tipo	Descripción
`ConfirmaA`	Propiedad	El valor del campo se completa solo a partir de la opcion elegida en otra lista.
`Positivo`	Método	Exige que el numero sea mayor que cero.
`Con(System.Object)`	Método	Define el valor inicial del campo (por ejemplo, la opcion elegida de un combo).

9.34.3. Dialogo

Cuadro de dialogo generico armado a partir de una lista de campos.

9.34.4. DialogoCobro

Registra un cobro de factura. Con efectivo no se pide banco ni referencia: esos datos se piden solo si el medio de pago los requiere (cheque, transferencia, tarjeta).

9.34.5. EditorLineas

Formulario para armar las lineas de un presupuesto o de una orden de compra.

Miembro	Tipo	Descripción
`PonerItems(Generic.IEnumerable{Ui.ItemLinea})`	Método	Vuelve a cargar lineas ya armadas (para no perderlas si el guardado falla).
`ModificarLinea`	Método	Cambia cantidad, precio y descuento de la linea elegida (y la descripcion si es fabricacion especial).

9.34.6. Estilo

Colores, fuentes y utilidades comunes de la interfaz.

Miembro	Tipo	Descripción
`Ejecutar(Forms.IWin32Window,System.Action)`	Método	Ejecuta una accion de negocio mostrando los errores de forma amigable. Devuelve true si termino sin error.

9.34.7. FilaCarga

Fila de la verificacion de carga: lo que se factura y se envia, y si ya se controlo en el camion.

9.34.8. FilaFaltante

Fila editable de la ventana de faltantes: lo que falta y la cantidad a ingresar.

9.34.9. FrmEspera

Ventana modal con barra de progreso que ejecuta un trabajo en un hilo aparte.

Miembro	Tipo	Descripción
`Ejecutar(System.String,System.Func{System.String})`	Método	Ejecuta el trabajo mostrando la ventana de espera. Devuelve el texto que produjo el trabajo y vuelve a lanzar su excepcion si fallo.

9.34.10. FrmFaltantes

Lista en rojo de los productos que faltan, con un contador por fila para ingresar el stock. Se usa al confirmar un pedido: se agrega lo que falta y se sigue sin salir de la pantalla.

Miembro	Tipo	Descripción
`Ingresos`	Propiedad	Cantidades elegidas para ingresar, por producto.

9.34.11. FrmVerificarCarga

Muestra lo que va a salir en el remito y obliga a verificarlo linea por linea al cargar el camion. Las cantidades vienen de lo facturado y reservado: no se pueden modificar.

Miembro	Tipo	Descripción
`Lineas`	Propiedad	Lineas que van en el remito, con la cantidad exacta que se facturo y esta lista.

9.34.12. GeoFicticio

Miembro	Tipo	Descripción
`Costa`	Campo	Contorno aproximado de la costa del Rio de la Plata (latitud, longitud), de oeste a este; el agua queda al norte y al este de esa linea.
`Ubicar(System.String,System.String)`	Método	Busca en la direccion (y el nombre del cliente) alguna localidad conocida. Si no encuentra ninguna, ubica el punto cerca de la Capital con un desplazamiento fijo calculado a partir del texto, marcandolo como estimado.

9.34.13. GrillaMarca

Grilla de datos que, si sobra espacio debajo de las filas, muestra el emblema como marca de agua.

9.34.14. Lugar

Ubicacion aproximada de localidades argentinas para dibujar el mapa de recorrido. Es un mapa esquematico y ficticio: no usa servicios externos ni claves de API.

9.34.15. MapaRuta

Mapa esquematico e interactivo del recorrido: sede en la Capital y paradas numeradas. Rueda del mouse para acercar, arrastrar para mover, doble clic para ajustar la vista.

9.34.16. MapaWeb

Mapa interactivo de la hoja de ruta: Leaflet con los mapas de OpenStreetMap dentro de WebView2. No necesita clave de API. Si el equipo no tiene WebView2 o no hay internet, muestra el mapa esquematico propio (MapaRuta).

Miembro	Tipo	Descripción
`UsaMapaWeb`	Propiedad	Indica si se esta usando el mapa web (si es falso, se ve el mapa esquematico).

9.34.17. Marca

Marca visual de Edelstahl: emblema (hexagono de acero con la E), textura de acero cepillado y paneles decorativos. Todo se dibuja por codigo: no usa fotos ni archivos externos.

Miembro	Tipo	Descripción
`Emblema(System.Int32)`	Método	Emblema cuadrado: hexagono de acero con una E grabada.
`Acero(System.Int32,System.Int32,Drawing.Color,Drawing.Color,System.Int32,System.Single)`	Método	Acero cepillado: degradado vertical con vetas horizontales finas.
`MarcaDeAgua(Drawing.Graphics,Drawing.Rectangle,System.Int32,System.Single)`	Método	Dibuja el emblema con transparencia, por ejemplo como marca de agua.

9.34.18. ModuloBase

Modulo de la aplicacion: un conjunto de pestanas, cada una con su lista. Una pestana solo aparece si el usuario tiene el permiso requerido.

9.34.19. Nombres

Titulos en ingles de las columnas y de los permisos. El texto en ingles es la clave que luego se traduce con los archivos de idioma.

9.34.20. Opcion

Elemento de lista desplegable con texto visible y valor asociado.

9.34.21. PanelAcero

Panel de acero cepillado con el emblema; se usa en la parte libre de la barra lateral.

9.34.22. PanelEncabezado

Encabezado de la ventana: acero claro con una linea de acento.

9.34.23. PanelLista

Lista con barra de acciones, busqueda, detalle opcional y exportacion. Los datos se leen en un hilo aparte para no congelar la ventana.

Miembro	Tipo	Descripción
`FilaCambiada`	Evento	Se produce cuando cambia la fila elegida (o llegan datos nuevos).

`Agregar(System.String,System.Action{System.Object},System.Boolean,System.Nullable{Security.Permiso})`	Método	Agrega un boton a la barra. Si requiereSeleccion es true solo se habilita con una fila elegida.
`HabilitarSi(Forms.Button,System.Func{System.Object,System.Boolean})`	Método	Habilita el boton solo cuando la fila elegida cumple la condicion.
`PintarFilas(Drawing.Color,Drawing.Color)`	Método	Tine todas las filas (por ejemplo en rojo para los faltantes).

9.34.24. Respuesta

Valores devueltos por el cuadro de dialogo.

9.34.25. Tr

Acceso corto a las traducciones. La clave es el texto en ingles.

Miembro	Tipo	Descripción
`Columna(System.String)`	Método	Convierte un nombre de propiedad en un titulo de columna (por ejemplo "FechaEmision" pasa a "Fecha Emision" y luego se traduce).